Learned Page Replacement in xv6

Week 15 update: new workloads, a wider model search, and the kernel

Summary

Week 14 showed that a small linear model over signals the kernel can see (accessed and dirty bits and cheap bookkeeping) beats Clock by 10–15% in faults on our benchmarks. This week tested how far that result reaches. All numbers are faults relative to Clock (below 1 is better), averaged geometrically over memory sizes of 5, 10 and 20% of each program's pages.

- **More, harder workloads.** Added the fixed `lzw` benchmark to the full pipeline, a synthetic pattern suite with known answers (`patbench`), replays of real Twitter cache traces, real Redis and SQLite page traces, and four application-shaped programs (pointer chase, sparse matrix–vector product, hash join, Bloom filter). Every program's access string is checked reference by reference against an independent model.
- **Literature baselines.** ARC, S3-FIFO, an MGLRU analogue, LHD and exact MRU are implemented in the simulator and checked against independent reference implementations. LHD is the strongest untrained policy (0.90–0.93 on our data); ARC and S3-FIFO are mixed, and S3-FIFO is worse than Clock on graph, matmul and sort.
- **Wider model comparison.** Trees (CART, random forest, boosted trees, ranking boosted trees, logistic regression) and neural nets (deeper MLP, set-attention, CNN and LSTM over page history, a CNN over the kernel's own scan-bit history, mixed models) were fitted on every scope and selected on validation only. With signals the kernel can actually observe, **no model class beats the linear scorer by the 2% and at-most-2×-cost bar overall**. Gains of 5–12% appear on single workloads (kv, the Twitter replays); models given oracle information gain up to 6% in distribution and fail badly on the real Redis and SQLite traces.
- **In the xv6 kernel** (157 runs on the new workloads, plus 61 with sampled scoring): the learned policy cuts faults by 16% on lzw (0.84), 15% on the Twitter replays (0.85) and 15% on patbench (0.85) against Clock, and does nothing on the application programs (1.00). The simulator predicts kernel faults to within about 4%, and the two larger gaps are explained (Section 9). Sampled scoring, now in the kernel, roughly halves the selection cost for 1–2.5% more faults.
- **Five extra models were ported to the kernel and their time delay measured** (Section 7). On kv a CNN over the aging bits and a deeper MLP are about 2.5% better than the linear model, but cost 85 and 148 times its instructions per eviction (1.7 and 2.9 million instructions against 19 000); boosted trees equal it at 21 times; on graph the linear model is clearly better. Timing is repeatable to under 0.3% because QEMU's clock is driven by instruction count.
- **AU5TRA's programs were merged** (pointer chase, hash join, Bloom filter, sparse matrix–vector, heap churn; Section 8): in our kernel each reproduces her model's trace exactly. The linear model gains most on heap programs (kernel 0.65 on `heap-mixed`) and fails on SpMV.
- **All three memory sizes were run in the kernel** (Section 10; long runs split over this machine and four Kaggle notebooks, whose results matched this machine exactly on all 261 runs done on both). Over nine workloads the linear model scores 0.94, 0.92 and 0.79 of Clock at 5, 10 and 20% memory; FIFO, Aging and LFU are worse than Clock at every size.
- **The two unexplained gaps are explained** (Section 9): the kernel starts from filled memory while the simulator started empty, and set-attention's result depends strongly on what is in memory at the start (up to 14 times), so its good simulated result was a lucky case.
- **Cost against number of candidates, and the break-even** (Section 11): the linear model pays for itself on a hard disk on every workload with a model of its own and about breaks even on fast flash;

the CNN, MLP and set-attention do not pay on either, because they cost about 75 to 145 times as much per eviction as the linear model.

- **Fairer comparison of the extra models.** Trained on all workloads, as the global linear model is, the CNN and MLP beat it by only 1.8% and 0.9% in the simulator (24 cells), below the 2% bar, at 75 to 145 times the cost.
- **A reproducibility bug was found and fixed.** Training samples depended on Python's per-process string hashing, so datasets could not be rebuilt identically. All models were refit; Clock, LRU and the other classical results are unchanged.

1 What changed in the data and the pipeline

Reproducibility fix. The sampling seed of each training stream used Python's `hash()`, which is randomised per process, so two builds of the same dataset differed. The seed is now a CRC32 of the stream name, a capacity and an optional salt. Rebuilding twice is bit-identical (checked over 11 dataset files); all 4 272 classical result rows of the original streams are unchanged. Because the old caches could not be recovered, every model was refit, so numbers move slightly from week 14 (for example the global default kernel model now has five features instead of four). The noise floor from sampling alone is measured in Section 13.

lzw. The corrected lzw benchmark from the team (`lzw-r20` for training, validation and test; `lzw-r30` held out) now runs through the whole pipeline: simulator sweep, neural models, selection, test, kernel export.

2 New workloads

workload	what it does	why it is in the study
`patbench`	11 synthetic modes over 1 024 pages: cyclic loop, scan with a hot set (two sizes), Zipf with skew 0.6–1.2, uniform, hot set that moves (two speeds), loop/Zipf switching	Known answers: a loop defeats LRU, FIFO and Clock and is solved by MRU; `uniform` has nothing to learn, so any gain there is a leak
`kv-T` (Twitter)	kvbench driven by real key streams from six production cache clusters, keys hash-sampled to kvbench's 1 024-key space (4 700–40 000 operations)	Real popularity and write mix; whole clusters are held out
real traces	Redis and SQLite page-touch strings collected on a host (12.3M and 8.6M references, no read/write marks)	Programs we did not write; tests transfer to a different kind of recording
`appbench`	pointer chase (mcf-like), CSR sparse matrix–vector, hash join, Bloom filter, each with an explicit multi-array layout	Irregular, scan-plus-random structure the other programs lack

Table 1: Workloads added this week. All are held out of training: they only test.

How strings are validated. Each program's reference string has an exact host-side model. The kernel's trace equals the model line for line for all 11 patbench modes (24 short runs and 9 full runs), and for all four appbench modes (30 000 references each, with matching read, write and checksum counters). The remaining production traces were then generated from the model rather than by booting xv6 for 17–45 minutes each; their logs carry a marker saying so and no kernel fault counters. The Twitter streams and the real traces are not generated: kvbench ran them in the kernel, and the Redis/SQLite files are the host recordings.

3 Literature baselines

Implemented in the simulator and cross-checked against independent reference implementations on 475 cases (identical counts): ARC (integer adaptation), S3-FIFO (as published, and a form that reads accessed bits only at queue heads), an MGLRU analogue for anonymous memory (generations, aging walk, rescue at eviction), LHD in a scan form, and exact MRU. MRU equals Belady on cyclic loops (9 of 9 checks), and

workload	variants	streams	references	held-out streams
app	4	12	12.0M	12
bloom	2	2	4.5M	2
btree	5	25	7.1M	5
chase	3	3	6.0M	3
graph	4	14	77.9M	3
heap	3	3	10.0M	3
join	3	3	2.3M	3
kv	13	41	8.5M	11
lzw	2	12	61.4M	6
matmul	6	6	13.8M	2
pat	11	55	55.0M	10
patb	10	10	20.0M	10
redis	1	1	12.3M	1
sort	3	9	16.6M	3
spmv	2	2	8.4M	2
sqlite	1	1	8.6M	1

Table 2: The study dataset after this week (176 streams).

LRU, FIFO and Clock miss every access of a loop larger than memory, as theory says. Table 4 shows them next to the learned models.

4 Transfer to unseen workloads

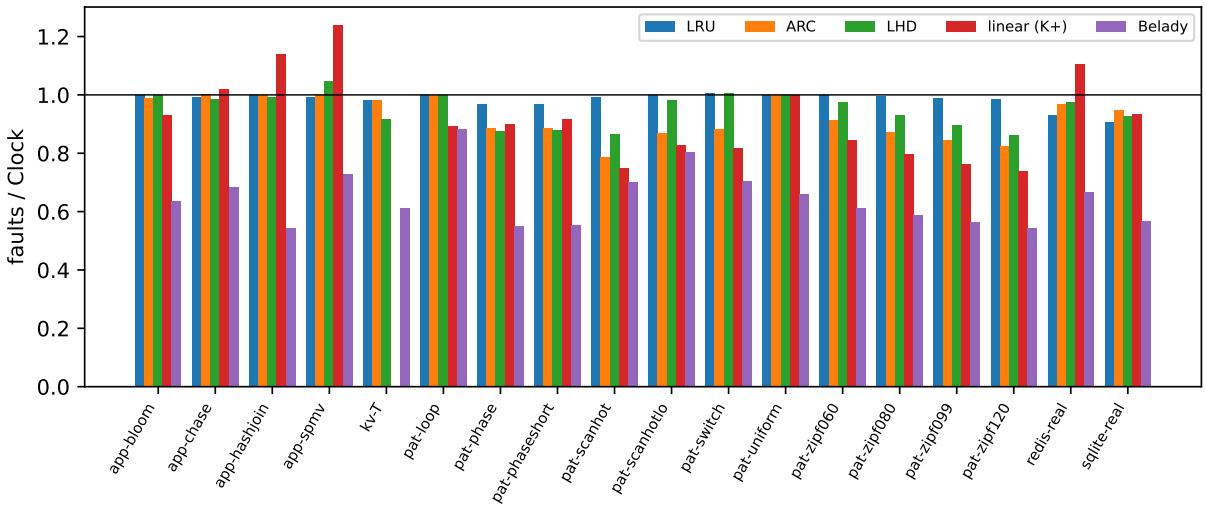


Figure 1: Faults relative to Clock on workloads no model was trained on. Linear (K+) is the global linear model over all kernel signals.

- **patbench known answers hold.** On `loop` FIFO, LRU, Clock, ARC and LHD miss everything and MRU matches Belady; on `uniform` every online policy is within 0.0003 of $1 - C/P$, so `uniform` is a clean leak control: no model gains there.
- The linear kernel-signal model beats ARC, S3-FIFO and LHD on the skewed and scan-plus-hot patterns (for example 0.76 against 0.85, 0.81 and 0.90 on Zipf 0.99) and on the Twitter replays (in the kernel, Section 6).
- **Where it loses.** Moving-hot-set patterns, SpMV and hash join. The oracle-feature variant (F+K) raises faults 2.6-fold on the moving hot set and reaches the 3×-Clock cap on Redis. The global kernel-signal linear model is 24% and 14% worse than Clock on SpMV and hash join. Real Redis/SQLite traces carry no write information, so dirty-bit signals are absent there, and LRU (0.93, 0.91) is the best simple policy.

stream	LRU	ARC	S3-FIFO	LHD	Belady	linear K+	linear F+K	linear kv K+
app-bloom	1.000	0.989	0.962	0.998	0.635	0.930	0.926	–
app-chase	0.991	1.002	1.021	0.984	0.683	1.021	1.020	–
app-hashjoin	0.998	1.000	1.012	0.994	0.544	1.138	1.017	–
app-spmv	0.992	1.000	1.026	1.046	0.727	1.239	0.995	–
kv-T	0.983	0.983	1.028	0.916	0.611	–	–	0.980
pat-loop	1.000	1.000	1.000	1.000	0.882	0.894	0.887	–
pat-phase	0.968	0.887	0.899	0.876	0.552	0.899	2.644	–
pat-phaseshort	0.967	0.888	0.909	0.879	0.552	0.918	2.686	–
pat-scanhot	0.992	0.786	0.795	0.866	0.700	0.748	0.744	–
pat-scanhotlo	1.001	0.870	0.874	0.982	0.803	0.829	0.903	–
pat-switch	1.005	0.882	0.877	1.007	0.704	0.817	0.805	–
pat-uniform	1.000	1.000	1.000	1.000	0.658	1.000	1.000	–
pat-zipf060	0.998	0.915	0.881	0.975	0.612	0.845	0.824	–
pat-zipf080	0.994	0.874	0.837	0.932	0.587	0.796	0.773	–
pat-zipf099	0.990	0.846	0.806	0.897	0.565	0.764	0.737	–
pat-zipf120	0.984	0.825	0.783	0.863	0.544	0.737	0.710	–
redis-real	0.929	0.967	1.104	0.973	0.667	1.106	3.000	–
sqlite-real	0.908	0.947	1.082	0.927	0.567	0.934	1.318	–

Table 3: Faults / Clock per stream group (geometric mean over streams and the three memory sizes). Simulator results; the three last columns are linear models (global kernel signals, global oracle+kernel signals, kv-scope kernel signals).

5 Wider model comparison

Protocol. The grid was written before any result: kernel-observable inputs (K) or all inputs (FK), with or without candidate-set rank features; CART (depth 4, 8), random forest, boosted trees (30 and 100 trees, 8 and 16 leaves), ranking boosted trees, logistic regression; eight neural architectures. Candidate sets come from the same recorded eviction decisions as before; the target is the log of the next-use distance. Probation is fixed at 2 scans. Selection is on validation streams at 10% memory only; the chosen models are then replayed on test, held-out and transfer streams at 5, 10 and 20%. Intervals are 95% stream-level bootstraps. The neural models run as numpy callbacks in the simulator (checked against PyTorch to 10^{-6}) and are not run on streams of more than 400 000 references for cost; the C tree scorer is checked against scikit-learn and LightGBM to 3×10^{-6}. A follow-up added four- to sixteen-tree boosted ensembles to meet the cost bar.

scope	split	LRU	ARC	S3F	LHD	linear K	tree K	NN K	NN K†	tree FK	NN FK	Belady
btree	test	0.994	0.950	0.862	0.948	0.835	0.834	0.822	0.817	0.829	0.811	0.550
btree	held-out	0.997	0.975	0.908	0.974	0.875	0.867	–	0.851	0.853	0.847	0.615
graph	test	0.990	0.991	1.161	0.865	0.847	0.847	–	0.844	0.806	0.844	0.456
graph	held-out	0.992	0.993	1.256	0.816	0.798	0.765	–	0.760	0.742	0.760	0.482
kv	test	0.984	0.986	1.047	0.906	0.897	0.902	0.850	0.849	0.875	0.843	0.559
kv	held-out	0.982	0.982	1.013	0.901	0.902	0.903	0.845	0.858	0.900	0.854	0.577
lzw	test	0.997	0.968	0.889	0.892	0.821	0.933	–	0.809	0.933	0.803	0.466
lzw	held-out	0.981	0.957	0.866	0.887	0.813	1.068	–	0.841	1.068	0.812	0.409
matmul	held-out	0.879	0.875	1.508	1.036	0.671	0.803	0.868	0.868	0.899	0.868	0.617
sort	test	1.012	1.013	1.321	1.008	0.983	0.879	–	0.891	0.854	0.901	0.830
sort	held-out	0.999	0.999	1.084	1.019	0.933	0.928	–	0.961	0.938	0.924	0.896
global	test	0.988	0.970	0.960	0.925	0.870	0.890	0.889	0.850	0.842	0.841	0.555
global	held-out	0.982	0.982	1.013	0.901	0.891	0.898	0.930	0.910	0.885	0.886	0.577

Table 4: Faults / Clock on test and held-out streams. "linear K" is the week-14 model class (kernel signals); "tree K" and "NN K" use kernel signals only; "NN K†" also reads the oracle reuse history; "FK" adds oracle inputs. A dash: not run (cost rule) or not selected.

What the wider comparison says.

- **Richer models help only where they are given information a kernel does not have.** With oracle inputs the best tree and neural models are 2–5% better than the best linear model in distribution; with kernel inputs only, trees tie the linear model (btree 0.834 against 0.835, graph held-out 0.765 against 0.798, global 0.890 against 0.870) and small boosted ensembles (4–16 trees) are *worse*: lzw 1.19 and 1.57 against 0.82 and 0.81 for the linear model.
- The deployable neural nets (deep MLP, set-attention, aging-bit CNN) gain 5–6% on kv and 2% on btree and lose 2–4% globally. Their kv gain is the one result that passes the 2% bar on a workload scope, at roughly 100× the arithmetic of a linear scorer.
- **Robustness is the real difference.** On the Twitter and application streams the kernel-input trees beat the linear model on the Twitter replays by 12% (0.86 against 0.98) and on the application streams by 3.5% (1.04 against 1.08), because they do not extrapolate the way a linear score does; but on the

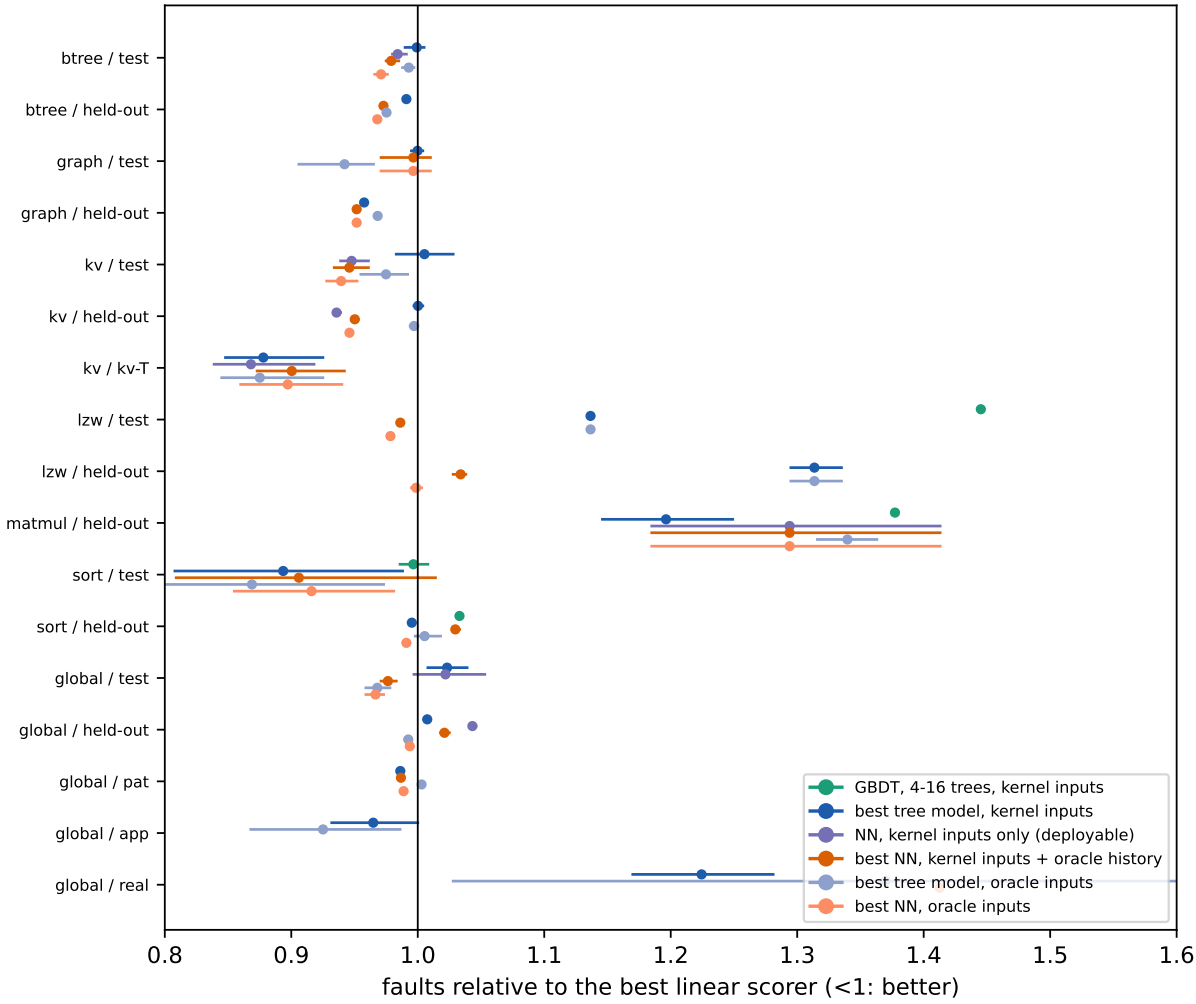


Figure 2: Faults relative to the best linear scorer (below 1: the model is better), with 95% intervals. Models with oracle inputs (blue-grey, orange-red, "oracle") cannot run in a kernel.

real Redis and SQLite traces every complex model is 22–41% worse than the linear one. None of the extra capacity is free of this trade.

- matmul has no validation streams (selection used its training streams), and every richer model is 20–30% worse than the linear one on its held-out streams: more capacity overfits when there is little data.

6 In the xv6 kernel

The refit linear models were re-exported to integer weights and the learned policy ran in the real kernel on the new workloads (patbench and appbench for 50 000 accesses; lzw and the Twitter replays in full) at 10% memory, all policies, same frame counts as the simulator.

- **The simulator predicts the kernel.** Kernel over simulator faults, geometric mean per policy: 0.98 on lzw, 0.96–0.98 on patbench and appbench (prefix replay), and 0.98–1.01 for the Twitter replays with the global and kernel-only models. The differences come from the kernel's own pages (code, stack) and capacity cliffs.
- **One exception, now explained.** For the kv-scope linear model on the Twitter replays the kernel was about 10% better than the simulator (0.90). The kernel's benchmarks fill their memory before the traced part starts and the simulator started empty; starting the simulator from filled memory brings the ratio to 1.00 (Section 9).

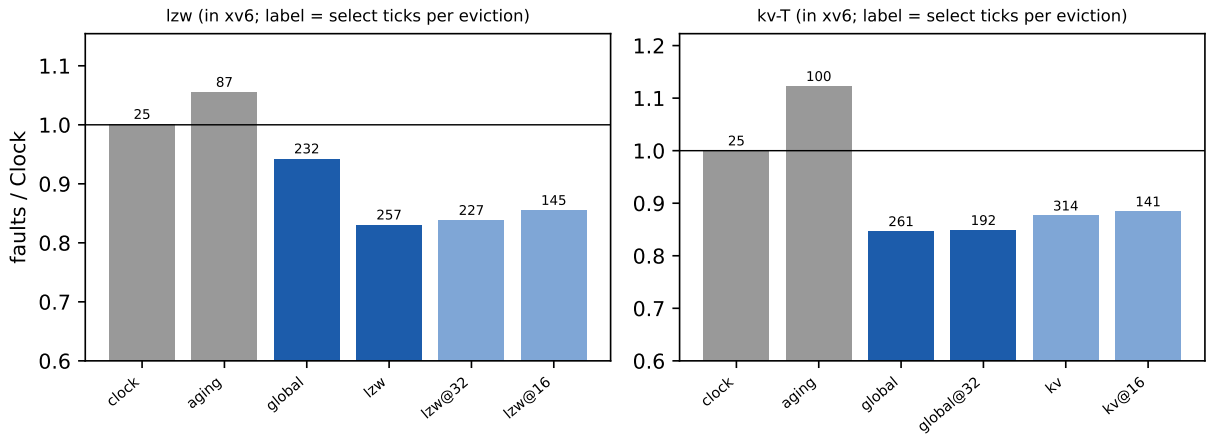


Figure 3: Kernel faults relative to Clock; the number on each bar is the selection cost in timer ticks per eviction (Clock: 24). "@k" scans and scores only k random candidates per eviction.

- The learned policy saves 16% of faults on lzw (per-workload model; 6% for the global model), 15% on the Twitter replays and 15% on patbench, and does not help on the application programs.
- **Cost, and sampled scoring.** The full scorer costs 170–430 ticks per eviction against 24 for Clock (7–17×). Scanning and scoring a random sample of 32 candidates takes 40–80% of that (136 ticks on lzw, 157 on the Twitter replays, 182 on patbench) for 0–1% more faults; 16 candidates takes 30–55% for 1–2.5%. The sampler is a partial Fisher–Yates over the candidate array and needs no extra memory.

7 The five extra models in the kernel

The extra models that use only kernel-visible signals were ranked on validation data (no test data used). The best of each of five kinds were converted to whole-number arithmetic and put into the kernel as built-in models, chosen with `vmrun ml <name>`: a CNN over the aging bits (trained on kv), set-attention (graph), a deeper MLP (kv), ranking boosted trees (graph) and boosted trees (kv). The integer versions choose the same victim as the original models on 381–399 of 400 recorded decisions, and give the same faults in the simulator to within 0.5% (one exception: ranking trees on one graph stream, 13% more). In xv6 they were run on every kv and graph test and held-out stream and the Twitter replays at 10% memory.

Time delay. The kernel now counts, per eviction, the time to build the candidate list, to choose the victim and for the whole eviction without the wait for the swap-out write. QEMU is run in a mode where its clock advances with the instructions executed (`QEMU_ICOUNT=1`), so the numbers do not depend on how busy the host is: two runs of the same policy differ by under 0.3% (0% for most). One run at a time, same stream and memory size for every policy.

- **On kv, the CNN and the MLP are about 2.5% better than the linear kv model** (0.849 and 0.853 against 0.871 on test; 0.850 and 0.848 against 0.878 held-out), less than the 5–6% the simulator predicted, and cost 85 and 148 times the linear model's instructions to choose. The boosted trees equal the linear model (0.871) at 21 times the cost.
- **On the Twitter replays** the three kv models are 0.853–0.860 against 0.876 for the linear kv model and 0.846 for the global linear model, so the global linear model is as good as the expensive ones there.
- **On graph the linear model wins clearly**: 0.618 (test) and 0.541 (held-out) against 0.697 and 0.676 for the ranking trees and 1.145 and 0.917 for set-attention, which is worse than Clock on test.
- **Kernel against simulator.** For the CNN, MLP and trees the kernel agrees with the simulator to 0.4–2.4% on kv and to 1% on the replays, so the conversion and the kernel code are right. On graph the kernel and simulator disagree more (ranking trees 1.06, set-attention 1.59 times the simulator's faults), but graph is noisy for every policy (Clock itself is 0.85 times its simulated faults) because of capacity cliffs. The poor set-attention result in the kernel is explained in Section 9: the model is unstable.
- **Cheaper linear scoring is the better trade.** Scoring 16 sampled candidates costs 0.3 times the full linear model's instructions for 1–2.5% more faults.

workload	policy	runs	faults / Clock	select ticks / eviction
lzw	fifo	2	1.132	9
lzw	clock	2	1.000	25
lzw	aging	2	1.054	87
lzw	lfu	2	1.064	84
lzw	ml global	2	0.942	232
lzw	ml lzw	2	0.830	257
lzw	ml lzw@64	2	0.830	306
lzw	ml lzw@32	2	0.837	227
lzw	ml lzw@16	2	0.854	145
kv-T	fifo	6	1.315	9
kv-T	clock	6	1.000	25
kv-T	aging	6	1.123	100
kv-T	lfu	6	1.180	97
kv-T	ml global	6	0.846	261
kv-T	ml global@32	6	0.849	192
kv-T	ml kv	6	0.876	314
kv-T	ml kv@32	6	0.875	223
kv-T	ml kv@16	6	0.884	141
pat	fifo	7	1.195	11
pat	clock	7	1.000	23
pat	aging	7	1.143	159
pat	lfu	7	1.148	156
pat	ml global	7	0.853	440
pat	ml global@32	7	0.861	213
pat	ml global@16	7	0.873	159
pat	ml global-k	7	0.950	377
app	fifo	4	1.219	11
app	clock	4	1.000	24
app	aging	4	0.982	124
app	lfu	4	1.091	122
app	ml global	4	1.004	348
app	ml global@32	4	0.996	186
app	ml global@16	4	0.993	128
app	ml global-k	4	0.997	311

Table 5: In-kernel runs (faults = zero-fill plus swap faults; geometric means over streams).

policy	kv test	kv heldout	kv-T	graph test	graph heldout
FIFO	1.280	1.310	1.315	1.305	1.181
Aging	1.106	1.133	1.123	1.173	1.046
linear, global	0.874	0.885	0.846	0.890	0.894
linear, kv	0.871	0.878	0.876	–	–
linear, graph	–	–	–	0.618	0.541
CNN on aging bits (kv)	0.849	0.850	0.853	0.903	0.984
deeper MLP (kv)	0.853	0.848	0.860	0.894	0.965
boosted trees (kv)	0.871	0.903	0.853	0.939	0.965
set-attention (graph)	1.755	1.611	1.859	2.194	1.400
ranking trees (graph)	0.973	0.874	1.320	1.011	1.017

Table 6: Faults relative to Clock in xv6 (geometric mean over streams). Each new model is shown only on the workload it was trained for.

8 Programs from AU5TRA

AU5TRA's branch was merged into this one. It adds the sources of five programs: pointer chasing (`chasebench`: a linked list, and a tree with 64- or 256-byte nodes), hash join (`joinbench`: uniform keys, Zipf keys, and a four-way partitioned join), a Bloom filter (`bloombench`), sparse matrix–vector product (`spmvbench`: random and banded), and heap allocation and release (`heapbench`: small, mixed and churn). It also adds an independent implementation of the patterns I had called patbench, with different strings; it is kept as `patbench2` and its streams are called `patb-*`. Her sweep folder `traces3/` held 116 traces, but only

policy	instructions to choose	x linear (kv)	instructions, whole eviction
FIFO (floor)	327	0.02	6,658
Clock	513	0.03	6,744
Aging	6,870	0.35	13,250
LFU	6,776	0.35	13,182
linear, global	18,512	0.95	24,650
linear, kv	19,422	1.00	25,538
linear, kv, 32 sampled	11,542	0.59	17,654
linear, kv, 16 sampled	5,824	0.30	11,968
CNN on aging bits	1,658,487	85.39	1,664,638
deeper MLP	2,868,034	147.67	2,874,219
boosted trees	416,962	21.47	423,127
set-attention	1,523,492	78.44	1,529,675
ranking trees	410,089	21.11	416,099

Table 7: Instructions per eviction on one kv stream (57 candidates per eviction). FIFO scans nothing, so it is the floor: the timer reads and the surrounding eviction code. At one nanosecond per instruction, 1 658 487 instructions is 1.7 ms.

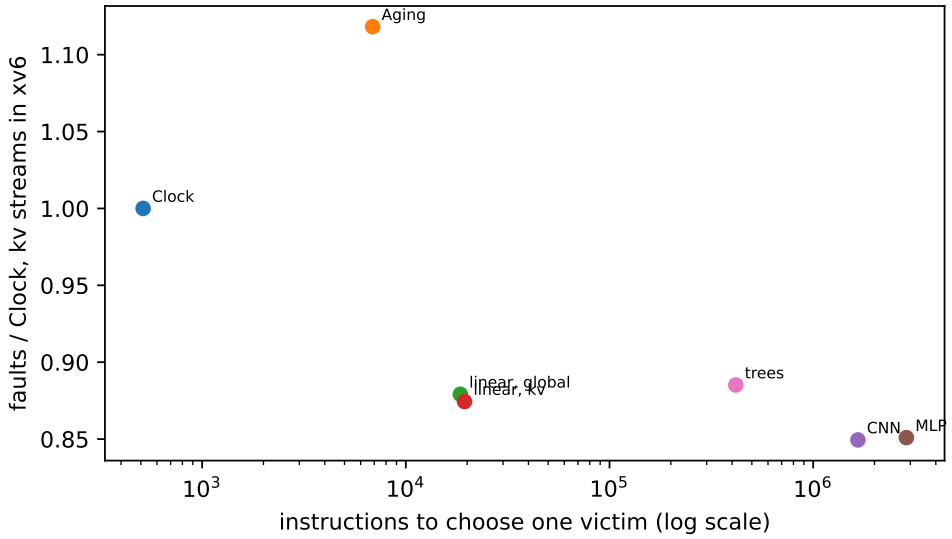


Figure 4: Fault reduction against the cost of choosing a victim on the kv streams.

26 different reference strings, because the string of a program does not depend on the memory limit; the other 90 are copies, and all of her btree, graph, kv, lzw, matmul and sort strings equal ones we already had.

Checks. Her host models reproduce the traces she collected in the kernel exactly (heap, join, SpMV). In our kernel each of the five programs produces a trace identical, byte for byte, to her model's prediction (short runs; the pointer-chase run is 149 870 references and the SpMV run 2 097 678).

Simulator results (23 streams, geometric mean of faults relative to Clock, over 5, 10 and 20% memory): the linear kernel-signal model 0.85, S3-FIFO 0.93, ARC 0.94, LHD 0.95; the five extra models 0.94 to 1.02. The linear model gains most on heap programs (0.62 to 0.82) and on hash joins with uniform or Zipf keys (0.73 to 0.80). Nothing helps on the pointer-chase list or the Bloom filter (all 1.00); the best possible policy, which sees the future, reaches 0.66 there, so there is room, but no online policy finds it. On her patbench the results agree with mine, for example 0.76 against 0.76 on Zipf 0.99, and the oracle-feature model fails in the same way on moving hot sets (2.4 times Clock).

In the kernel (13 streams at 10% memory, 143 runs, no failures) the global linear model scores 0.91 against Clock. The heap programs improve most (0.65 on `heap-mixed`). Two of the extra models are dangerous outside their training workload: on the banded SpMV, set-attention makes 47.7 times Clock's faults and the ranking trees 6.7 times.

9 Why the kernel and the simulator disagree

Two results of the first version of this report were unexplained. Both are now explained.

The kernel starts from filled memory; the simulator started empty. Every benchmark first touches all its pages (the populate step, not traced) and only then starts the traced part, so the traced part begins with a full memory whose contents the policy has already shaped. The simulator replayed the traced string from empty memory. Putting one sweep over the pages in front of the string (its own faults not counted) removes the kv-scope linear model's gap on the Twitter replays: kernel over simulator changes from 0.903 to 0.999. For the other policies the change is at most 0.02.

streams	policy	kernel / simulator, empty start	kernel / simulator, populated start
kv	Clock	1.016	1.018
kv-T	Clock	1.008	1.009
kv	linear, global	1.005	1.013
kv-T	linear, global	0.978	0.999
kv	linear, kv	0.991	1.010
kv-T	linear, kv	0.903	0.999
kv	CNN	1.016	1.015
kv-T	CNN	1.010	1.010
kv	MLP	1.014	1.016
kv-T	MLP	1.007	1.012
kv	boosted trees	1.006	1.007
kv-T	boosted trees	0.996	1.004

Table 8: Kernel faults over simulator faults (geometric mean over streams) with the simulator started empty and started from filled memory.

Set-attention is very sensitive to the starting state. I replayed 14 streams from five starting states (empty, and after a populate sweep in ascending, descending and two random orders) and took, for each policy and stream, most faults over fewest.

policy	streams	median spread	largest spread
clock	14	1.005	1.02
global	14	1.010	1.05
kv	3	1.117	1.14
graph	3	1.001	1.04
btree	1	1.011	1.01
lzw	1	1.020	1.02
sort	1	1.003	1.00
matmul	1	1.064	1.06
cnn-kv	14	1.003	1.04
mlp-kv	14	1.002	1.03
trees-kv	14	1.007	1.03
ranktrees-graph	14	1.015	1.37
attention-graph	14	2.009	14.40

Table 9: Spread of simulated faults over five starting states: median and largest over streams. 1.00 means the starting state does not matter.

Clock, the global linear model, the CNN, the MLP and the boosted trees change by 1–5% (largest spread 1.05). The linear model of one workload changes by more: matmul 6%, kv 12% (the gap above). Set-attention changes by a median of 2.0 times and up to 14.4 times (on one graph stream its faults go from 84 820 to 164 265), and the ranking trees by up to 1.37 times. In the simulator, from the empty start, set-attention looked excellent on graph (0.50 of Clock on five of the six streams and 0.78 on the sixth); in the kernel it scored 0.91 to 1.30. The empty start was the lucky case, and the kernel result lies inside the spread. A model whose result depends this much on what happened to be in memory first should not be trusted from one run.

10 All three memory sizes

The 10% runs cover every stream. At 5% and 20% memory one stream per program variant was run, 42 streams, with the standard policies and the linear models everywhere, and the five extra models on one stream per workload. 1 694 in-kernel runs were merged into one file. A run is identified by its stream, memory size, policy and model, and the long runs were split over this machine and four Kaggle notebooks. Where a run was done on both, the results are identical: all 261 runs done on both the Kaggle machines and this one gave the same faults, writes and evictions.

policy	btree	graph	kv	kv-T	lzw	matmul	sort	heap	join	spmv	chase	bloom
FIFO	1.15	1.06	1.41	1.52	1.19	1.73	1.40	1.09	1.10	1.09	1.15	1.11
Aging	1.09	1.02	1.07	1.08	1.02	1.72	0.98	1.00	1.03	0.99	1.08	1.00
LFU	1.11	1.03	1.13	1.23	1.04	1.72	0.99	1.00	1.03	1.04	1.09	1.00
linear, global	0.82	1.01	0.81	0.78	0.96	1.48	0.98	0.92	0.91	1.14	0.83	1.00
linear, own workload	0.86	0.97	0.81	0.80	0.89	1.97	0.97	0.92	0.91	1.14	0.83	1.00
CNN on aging bits	0.89	–	0.78	0.76	–	1.00	1.09	1.00	0.97	1.00	0.94	1.00
deeper MLP	0.89	–	0.79	0.77	–	0.99	1.04	1.00	0.97	1.01	0.93	1.00
boosted trees	0.88	–	0.79	0.77	–	0.99	1.13	0.99	0.97	1.01	0.92	1.00
set-attention	1.63	–	1.60	1.45	–	0.99	1.13	0.93	1.27	1.91	1.42	1.00
ranking trees	0.97	–	0.81	1.02	–	1.60	1.22	1.05	0.95	1.31	0.92	1.00

Table 10: 5% memory. Faults relative to Clock in xv6. Every row of a column averages the same streams (those the extra models were run on). The extra models were trained on kv or graph only, so their other columns show how they carry over. A dash: not run.

policy	btree	graph	kv	kv-T	lzw	matmul	sort	heap	join	spmv	chase	bloom
FIFO	1.13	1.86	1.28	1.31	1.13	1.82	1.28	1.04	1.04	1.63	1.09	1.06
Aging	1.11	1.50	1.11	1.12	1.04	1.00	0.99	1.02	1.01	1.00	1.06	1.00
LFU	1.12	1.81	1.16	1.18	1.05	1.00	1.00	1.02	1.01	1.58	1.06	1.00
linear, global	0.82	0.83	0.88	0.85	0.94	1.10	0.97	0.73	0.99	1.12	0.86	1.00
linear, own workload	0.83	0.60	0.87	0.88	0.85	0.97	0.95	0.73	0.99	1.12	0.86	1.00
CNN on aging bits	0.93	0.80	0.85	0.85	0.98	0.99	1.00	0.98	0.97	1.02	0.96	1.00
deeper MLP	0.97	0.62	0.85	0.86	1.07	0.98	1.02	1.00	0.98	1.02	0.94	1.00
boosted trees	0.89	0.99	0.88	0.85	0.97	1.01	1.07	0.93	1.00	1.10	0.97	1.00
set-attention	2.00	1.08	1.74	1.86	1.13	0.91	1.30	0.76	1.12	9.03	1.06	1.00
ranking trees	1.14	0.69	0.96	1.32	0.89	0.94	1.18	0.98	0.97	2.67	0.99	1.00

Table 11: 10% memory, same layout and the same streams.

policy	btree	graph	kv	kv-T	lzw	matmul	sort	heap	join	spmv	chase	bloom
FIFO	1.14	1.72	1.23	1.35	1.39	1.66	1.14	1.04	2.51	1.33	1.11	1.03
Aging	1.12	1.46	1.15	1.23	1.30	1.11	1.00	1.01	2.46	1.01	1.09	1.00
LFU	1.13	1.68	1.17	1.28	1.33	1.11	1.00	1.01	2.46	1.32	1.09	1.00
linear, global	0.91	1.60	0.94	0.83	0.87	0.98	0.90	0.69	0.41	0.81	0.80	1.00
linear, own workload	0.94	0.97	0.93	0.87	0.71	1.05	0.86	0.69	0.41	0.81	0.80	1.00
CNN on aging bits	0.94	–	0.89	0.81	–	0.95	1.00	0.95	0.52	1.00	0.94	–
deeper MLP	1.00	–	0.89	0.82	–	0.95	1.01	1.00	0.40	1.01	0.92	–
boosted trees	0.96	–	0.98	0.83	–	1.03	1.01	0.93	1.42	1.16	1.01	1.00
set-attention	2.18	–	2.42	3.20	–	0.91	0.86	0.68	6.82	13.62	0.99	–
ranking trees	1.33	–	0.98	1.58	–	1.07	1.11	1.00	0.81	5.26	1.08	1.00

Table 12: 20% memory, same layout and the same streams.

Geometric mean over the nine workloads for which every row has data (all except graph, lzw and Bloom), at 5, 10 and 20% memory:

policy	5%	10%	20%
FIFO	1.28	1.27	1.34
Aging	1.10	1.05	1.19
LFU	1.13	1.11	1.24
linear, global	0.94	0.92	0.79
linear, own workload	0.98	0.91	0.79
CNN on aging bits	0.93	0.95	0.87
deeper MLP	0.93	0.96	0.86
boosted trees	0.93	0.96	1.03
set-attention	1.34	1.58	2.11
ranking trees	1.07	1.17	1.31

- **The linear kernel-signal model beats Clock at all three sizes**, by 6%, 8% and 21%, and is the best of the rows at 10 and 20%. The CNN and MLP also beat Clock at all three (by 5–13%), but only by being run on workloads they were not trained for with about a hundred times the arithmetic. The 20% figure of the linear model is pulled down by the hash join, where it scores 0.41 against Clock; without the join the mean is 0.85.
- Where it fails: matmul at 5% (1.48, with only 2 frames), SpMV (1.14 and 1.12 at 5 and 10%), and, for the global model, graph at 20% (1.60). The graph model of its own is 0.97 at 5 and 20% against 0.60 at 10%: its gain is concentrated at one memory size, as week 14 found.
- The CNN, the MLP and the boosted trees are about 1% better than the global linear model at 5%, and worse at 10 and 20%, though they were trained on one workload only (see the table caption). Set-attention and ranking trees are worse than Clock at every size.
- FIFO, Aging and LFU lose to Clock by 4 to 34%.

11 What choosing a victim costs, and when it pays

The earlier timing was one stream and one memory size. Here the memory limit is varied: `patbench` in uniform mode over 1 024 pages, with 16, 64, 256 and 512 frames, so that every eviction chooses among that many candidates, for every policy. QEMU runs in instruction-count mode, so the counts do not depend on the host. The unit is instructions spent in the policy itself (the candidate list and the swap-out wait are not included).

policy	16 candidates	64 candidates	256 candidates	512 candidates
FIFO	145	385	1,345	2,625
Clock	404	411	646	1,069
Aging	2,461	8,125	30,778	60,979
LFU	2,436	8,005	30,279	59,982
linear, full scan	5,995	22,803	91,720	183,311
linear, 32 sampled	5,997	11,293	11,813	11,907
linear, 16 sampled	5,139	5,656	5,914	5,974
boosted trees	146,625	494,506	1,872,534	3,715,591
ranking trees	139,724	468,968	1,766,567	3,514,671
set-attention	536,987	1,811,720	6,833,988	13,529,003
CNN on aging bits	583,317	1,982,677	7,582,473	15,052,603
deeper MLP	1,006,459	3,421,490	13,083,639	25,971,866
CNN, 32 sampled	583,318	933,686	933,929	934,207
MLP, 32 sampled	1,006,460	1,610,766	1,611,008	1,611,358

Table 13: Instructions to choose one victim against the number of candidates. “Sampled” policies look at that many random candidates at most.

- The cost of a full scan grows in a straight line with the candidates: about 120 instructions per candidate for Aging and LFU, 360 for the linear model, 7 300 for boosted trees, 27 000 for set-attention, 30 000 for the CNN and 51 000 for the MLP. Clock costs under 1 100 instructions up to 512 candidates and FIFO under 2 700.
- Sampling 32 candidates makes the cost independent of the memory size: the linear model costs 11 800 instructions at 256 candidates and at 512, against 91 700 and 183 300 for the full scan. The CNN with 32 sampled still costs about 930 000.
- The rest of an eviction (finding the candidates and the bookkeeping) is about 6 000 to 6 500 instructions for every policy (measured on the kv stream, 57 candidates).

Does the saving in faults pay for the extra time? For each workload I took the faults saved per eviction (from the kernel runs) and the measured cost of the policy at that run’s number of candidates, and computed the *break-even*: the cost of one fault above which the policy wins. The time is then the faults times their cost plus the evictions times the choosing cost, relative to Clock. Two fault costs are shown: 100 µs (a fast flash disk) and 8 ms (a hard disk).

- **The linear model of its own workload pays for itself on a hard disk on every workload that has one** (btree, graph, kv, kv-T, lzw, sort: time 0.58 to 0.94 of Clock’s), and roughly breaks

workload	policy	faults / Clock	instructions to choose	break-even (µs per fault)	time / Clock, SSD	time / Clock, disk
btree	linear, own workload	0.84	82,398	527	1.509	0.848
btree	CNN	0.948	6,825,367	208,093	63.184	1.742
btree	MLP	0.974	11,776,868	670,968	111.236	2.375
btree	boosted trees	0.901	1,686,784	18,662	15.534	1.09
graph	linear, own workload	0.576	57,649	128	0.901	0.58
graph	CNN	0.873	4,807,623	18,310	41.808	1.394
graph	MLP	0.629	8,295,461	23,871	51.549	1.276
graph	boosted trees	0.959	1,189,284	11,688	12.097	1.101
kv	linear, own workload	0.873	20,236	149	1.045	0.875
kv	CNN	0.843	1,757,359	10,696	15.506	1.028
kv	MLP	0.844	3,032,548	18,385	26.191	1.164
kv	boosted trees	0.878	437,203	3,780	4.68	0.926
kv-T	linear, own workload	0.876	18,469	130	1.034	0.878
kv-T	CNN	0.853	1,613,266	10,424	14.541	1.024
kv-T	MLP	0.86	2,783,931	17,533	24.686	1.159
kv-T	boosted trees	0.853	401,675	2,689	4.261	0.896
lzw	linear, own workload	0.837	15,589	95	0.963	0.838
lzw	CNN	0.983	1,378,500	83,605	14.42	1.152
lzw	MLP	1.087	2,378,865	never	26.715	1.409
lzw	boosted trees	0.964	343,790	9,656	4.25	1.005
heap	CNN	0.984	2,371,744	113,534	20.777	1.264
heap	MLP	1.001	4,092,605	>1,000,000	35.644	1.484
heap	boosted trees	0.931	588,687	16,327	5.665	0.999
join	CNN	0.969	4,598,555	145,032	44.23	1.521
join	MLP	0.98	7,934,737	738,963	76.441	1.939
join	boosted trees	0.995	1,137,735	260,608	12.002	1.136
sort	linear, own workload	0.944	3,536	59	0.974	0.944
sort	CNN	1.0	395,813	21,055	4.839	1.049
sort	MLP	1.018	683,342	never	7.754	1.105
sort	boosted trees	1.056	101,496	never	2.106	1.07

Table 14: Break-even and time relative to Clock, at 10% memory (one instruction taken as one nanosecond).

even on fast flash (0.90 on graph, 0.96 on lzw, 1.03–1.05 on kv, 1.51 on btree, where 230 candidates make the scan expensive).

- The CNN and the MLP never pay: their break-even is 10 to 670 ms per fault, and they are 1.0 to 2.4 times slower than Clock on a hard disk and 5 to 111 times slower on flash. Set-attention costs about as much as the CNN and does worse in faults, so it cannot pay either.
- Boosted trees pay only on a hard disk, on kv (0.93 of Clock's time) and kv-T (0.90), and nowhere else.
- Sampling keeps the linear model's choosing cost from growing with the memory size.

How sure are the kernel numbers? A kernel run repeats exactly, so there is no run-to-run noise. What varies is the stream, so for each workload I resampled its streams (10 000 times) to get a 95% interval, and compared each extra model with the linear model on the same streams.

workload	streams	linear, own workload	linear, global	CNN	MLP	boosted trees
btree	9	0.844 [0.814, 0.866]	0.841 [0.811, 0.863]	0.948 [0.93, 0.964]	0.974 [0.962, 0.984]	0.901 [0.88, 0.916]
graph	6	0.578 [0.541, 0.658]	0.736 [0.587, 1.092]	0.873 [0.716, 1.038]	0.629 [0.606, 0.651]	0.959 [0.76, 1.255]
kv	11	0.874 [0.836, 0.907]	0.879 [0.839, 0.91]	0.849 [0.808, 0.885]	0.851 [0.811, 0.885]	0.885 [0.841, 0.92]
kv-T	6	0.876 [0.837, 0.929]	0.846 [0.799, 0.906]	0.853 [0.807, 0.914]	0.86 [0.815, 0.921]	0.853 [0.814, 0.899]
lzw	7	0.837 [0.831, 0.844]	0.941 [0.937, 0.944]	0.983 [0.981, 0.985]	1.087 [1.076, 1.095]	0.964 [0.963, 0.966]
sort	5	0.944 [0.933, 0.96]	0.967 [0.961, 0.974]	1.0 [0.994, 1.003]	1.018 [1.009, 1.033]	1.056 [1.035, 1.097]
heap	3	–	0.735 [0.65, 0.832]	0.984 [0.974, 0.99]	1.001 [0.997, 1.006]	0.931 [0.832, 0.991]
join	3	–	0.987 [0.916, 1.037]	0.969 [0.958, 0.991]	0.98 [0.96, 0.993]	0.995 [0.994, 0.996]
pat	11	–	0.873 [0.81, 0.944]	0.921 [0.865, 0.979]	0.922 [0.871, 0.972]	0.892 [0.829, 0.967]
app	4	–	1.004 [0.959, 1.05]	1.0 [0.985, 1.017]	1.004 [0.988, 1.018]	0.993 [0.956, 1.032]

Table 15: Faults relative to Clock in xv6 at 10% memory, with 95% intervals across the streams of each workload.

On kv the CNN and the MLP use 2.9% and 2.7% fewer faults than the linear kv model (intervals 0.966–0.977 and 0.968–0.979 over 11 streams); on the Twitter replays 2.7% and 1.9%, and so do the boosted trees (2.7%). Everywhere else they use more faults than the linear model. The gain on kv is small but clearly not noise.

12 A safety guard (prototype, simulator only)

The linear model loses to Clock on some workloads (SpMV, parts of matmul and graph). A guard should stop that. The prototype alternates the two policies on the same memory: one window of W accesses under the learned policy, one under Clock, compare the faults of the two windows, use the better one for the next

workload	model	streams	faults vs linear [95% interval]	result
btree	CNN	9	1.124 [1.09, 1.168]	more faults
btree	MLP	9	1.154 [1.117, 1.2]	more faults
graph	set-attention	6	1.772 [1.669, 1.914]	more faults
graph	ranking trees	6	1.187 [1.034, 1.287]	more faults
kv	CNN	11	0.971 [0.966, 0.977]	fewer faults
kv	MLP	11	0.973 [0.968, 0.979]	fewer faults
kv	boosted trees	11	1.012 [1.001, 1.024]	more faults
kv-T	CNN	6	0.973 [0.964, 0.984]	fewer faults
kv-T	MLP	6	0.981 [0.973, 0.99]	fewer faults
kv-T	boosted trees	6	0.973 [0.964, 0.982]	fewer faults
lzw	CNN	7	1.175 [1.166, 1.182]	more faults
lzw	MLP	7	1.299 [1.274, 1.316]	more faults
sort	CNN	5	1.059 [1.035, 1.076]	more faults
sort	MLP	5	1.079 [1.075, 1.083]	more faults

Table 16: Extra models against the linear model of the same workload, on the same streams, in xv6.

K windows, and test again. It runs in the simulator from a filled-memory start, with the global linear model as the learned policy. I tried window lengths of $n/200$ to $n/50$ of the stream and K from 4 to 80 on 11 streams (33 cells) and carried three settings over to 18 streams the grid had never seen (54 cells).

policy	all 33 cells	8 cells where learned loses	22 cells where it wins	worst cell
learned policy alone	0.913	1.133	0.834	1.491
guarded, window n/200, retest every 20 windows	0.924	1.013	0.882	1.036
guarded, window n/100, retest every 10 windows	0.926	1.014	0.885	1.045

Table 17: Streams the settings were chosen on (faults relative to Clock; "all cells" is the geometric mean).

policy	all 54 cells	6 cells where learned loses	34 cells where it wins	worst cell
learned policy alone	0.861	1.289	0.758	1.860
guarded, window n/200, retest every 20 windows	0.881	1.070	0.811	1.356
guarded, window n/100, retest every 10 windows	0.883	1.079	0.812	1.362

Table 18: Streams the settings were not chosen on.

- On the streams used to choose the settings the guard looks very good: the worst cell falls from 1.49 to 1.04 and the cells where the learned policy loses from 1.13 to 1.01.
- **On streams it was not tuned on it helps less**: the worst cell falls from 1.86 to 1.36, the cells where the learned policy loses from 1.29 to 1.07, and the average gain drops from 0.861 to 0.881 of Clock. The guard roughly halves the damage; it does not remove it.
- The guard hands about 40% of evictions to Clock even where the learned policy wins, because a window that follows a Clock window starts from memory that Clock has shaped, which makes the learned policy look slightly worse. A better test would remove that bias.
- *This is not in the kernel.* The kernel cannot count accesses, which the window length uses; a kernel version needs another measure of progress (for example time), which has its own problems because the learned policy's own running time would count against it. That choice is open.

The same models trained on all workloads. The comparison above is not fair to the extra models, because the global linear model was trained on every workload and they were not. So each of the five kinds was also trained on all workloads (the training workloads are the same as for the global linear model: btree, graph, kv, lzw, matmul, sort; the new programs are not in the training data). Simulator, filled-memory start, the whole-number versions the kernel uses, 8 streams at 5, 10 and 20% memory (24 cells), faults relative to Clock:

Trained on all workloads, the CNN (0.878) and the MLP (0.886) beat the global linear model (0.894) by 1.8% and 0.9%, less than the 2% bar, at 75 to 145 times the cost of choosing a victim. Boosted trees (0.963) and

model	trained on one workload	trained on all workloads
linear, global	–	0.894
CNN on aging bits	0.939	0.878
deeper MLP	0.926	0.886
boosted trees	0.981	0.963
set-attention	1.478	0.895
ranking trees	1.109	1.186

Table 19: Simulator, 24 cells: geometric mean of faults relative to Clock.

ranking trees (1.186) do not beat it. Set-attention improves from 1.48 to 0.90, level with the linear model, but it stays unstable (Section 9). The same five models trained on all workloads are built into the kernel (`cnn-all`, `mlp-all`, `trees-all`, `attention-all`, `ranktrees-all`); their kernel runs are *work in progress.*

13 Noise floor

The linear model was refit from three extra datasets built with different sampling seeds and replayed on the same test and held-out cells. The faults/Clock geo-mean of the global K+K+ and F+K models for each data seed is

feature group	faults/Clock per data seed	range
F+K	0.9452 / 0.9433 / 0.9444 / 0.9459	0.0026
K+K+	0.8942 / 0.8942 / 0.8945 / 0.8948	0.0005

Data sampling moves the result by at most 0.0005 (kernel signals) and 0.0026 (with oracle signals), under 0.3%, so the differences between models reported above (1–14%) are not artefacts of which training rows were drawn. It does not measure the other sources of variance (the choice of streams, which the bootstrap intervals cover).

14 Limits and what is next

- The real Redis/SQLite traces and the Twitter replays are small or lack read/write marks; the Twitter streams are hash-sampled to 1 024 keys, which preserves each key's request sequence but not the original working-set size.
- Neural models were not scored on streams of more than 400 000 references in the simulator, so the simulator comparison on those streams covers trees only; in the kernel all five models were run.
- The kernel runs of patbench and appbench use 50 000 accesses, not the full 1M, for time.
- The five extra models were each trained on one workload (kv for the CNN, MLP and boosted trees; graph for set-attention and ranking trees). Where they are run on other workloads (Table 10) the numbers show how well they carry over, not what they could do if trained there. Training them on all workloads was done in the simulator (Section 7, 24 cells); the kernel runs of those all-workloads models, and training on the new programs, are not finished.
- At 5% and 20% memory one stream per program variant was run (42 streams), not every seed. Three runs are missing: the CNN, MLP and set-attention on the Bloom filter at 20%, which hit the three-hour limit on the Kaggle machine.
- Time is counted in instructions, at one nanosecond per instruction, which is conservative for a real processor. The break-even figures depend on that and on the cost of a fault on the machine in question, which xv6 on QEMU does not give; two example costs are shown.
- The week-14 and the current builds of the benchmark programs differ, which moves every policy's fault counts by about 1%. Results were only compared within one build.
- Not done: training on the new programs; the Bloom filter 20% cells above; the guard in the kernel (Section 12); LHD in the kernel; a measured cost of a real page fault.

Reproducing

```
python3 tools/ml2/extra_trees.py fit | select | test      # tree and neural models (extra_nn.py fit first)
python3 tools/ml2/extra_report.py                          # extra models vs linear vs baselines
python3 tools/ml2/exp_transfer.py run [--set real|app]   # transfer streams
python3 tools/ml2/kernel_eval.py --only lzw,kv-T --fracs 0.1  [--models lzw@32]
python3 tools/ml2/merge_kernel_results.py                # one file of all in-kernel runs (kernel_eval_all.csv)
python3 tools/ml2/cost_grid.py; python3 tools/ml2/cost_benefit.py   # cost against candidates; break-even
python3 tools/ml2/state_sensitivity.py                   # dependence on the starting memory state
python3 tools/kaggle/make_package.py                     # split of the long kernel runs over Kaggle notebooks
python3 tools/ml2/kernel_vs_sim.py --only lzw,kv-T,pat,app --tag _w15
python3 tools/ml2/report_w15.py                          # tables and figures of this report
```